Introdução à Aprendizagem Automática 2026/2027

Aprendizagem Supervisionada - Mini Projeto de Laboratório II

18 de outubro de 2026

1 Introdução

Este mini-projeto tem como objetivo a familiarização prática com os algoritmos de aprendizagem supervisionada lecionados nas aulas teóricas, assim como com as boas práticas a adoptar na avaliação e seleção de modelos e seus hiperparâmetros. Em particular, é esperado que os alunos apliquem os conhecimentos adquiridos e os expandam seguindo um conceito de aprendizagem ativa, tendo por base um conjunto de dados reais. Este mini-projeto terá componentes de programação (implementação de algoritmos), aplicação dos algoritmos para visualização e interpretação/análise dos resultados. **As aulas de laboratório servirão para acompanhar e avaliar de forma contínua a progressão dos projetos, mas é esperado que os alunos dediquem tempo ao trabalho fora das aulas de laboratório.**

O trabalho deverá ser realizado em **grupos de 2 alunos**, sendo que cada grupo deve trabalhar sozinho. A linguagem de programação a usar será o **Python**. Recomenda-se a instalação de, pelo menos, as seguintes bibliotecas: *numpy, scipy, matplotlib, pandas, scikit-learn*. Alternativamente, os alunos poderão usar o **Google Colab**[*] como plataforma para implementarem todo o código do projeto. O Colab tem o benefício de que pode ser associado a uma conta de endereço Gmail, interliga-se ao Google Drive e não necessita da instalação das bibliotecas mais populares.

2 Submissão e Regras Sobre o Uso de IA

Cada grupo deve submeter um **ficheiro zip** no Fénix até dia **5 de Dezembro de 2026**, com os seguintes ficheiros:

- Um **ficheiro Python**, idealmente um *Jupyter Notebook*, contendo todo o código desenvolvido para o projeto. Código que faça parte da avaliação, nomeadamente que corresponda a perguntas para escrita de funções, deve estar claramente indicado com comentários.
- O **PDF do enunciado preenchido** com as respostas às questões nos campos indicados - isto pode ser feito preenchendo diretamente o PDF num browser ou num leitor de PDF. Posteriormente, o PDF deve ser *flattened*/trancado, o que pode ser feito escolhendo a opção de imprimir o PDF e posteriormente guardar novamente apenas as páginas pretendidas. **Não exceder o limite das caixas de texto porque é cortado após o flattenig**
- Uma pasta chamada **Figuras** com todos os **gráficos/figuras pedidas**, identificadas com o nome da questão (*e.g.*, "PartI2a.jpg"). Figuras extra podem ser incluídas e devem ser identificadas da mesma forma e referidas no texto. Os eixos e pontos dos gráficos devem ser sempre corretamente identificados.
- Os documentos do **compromisso de honra** e da declaração do uso de IA preenchidos e assinados. Ver templates dos mesmos nos anexos A e B.

[*] https://colab.research.google.com/

Uso de IA: Os laboratórios de Introdução à Aprendizagem Automática têm uma política de "semáforo amarelo"quanto ao uso de IA. Permite-se o uso da mesma para auxílio na geração de código e na compreensão de bibliotecas e funções das mesmas. Adicionalmente, é permitido o uso de IA como suporte à aprendizagem ativa. **Todavia, os alunos deverão ser capazes de responder a questões feitas sobre o código durante as aulas, sendo isto contabilizado na componente da avaliação contínua. Não é permitido o uso de IA para gerar respostas.**

3 Avaliação

A nota do mini-projeto II terá em conta o **código entregue**, as **respostas às questões** e a **avaliação contínua** a decorrer durante as duas sessões de laboratório dedicadas a este projeto. A avaliação contínua consistirá na interação com o docente de laboratório, sendo que a mesma poderá ter diferentes formatos. Este projeto terá um peso de 40% na nota final do **laboratório** da cadeira.

4 Dados

4.1 Descrição dos dados

O projeto tem como objetivo principal a análise supervisionada de vídeos de fisioterapia de vários pacientes que sofreram acidentes vasculares cerebrais (AVC). Os pacientes foram gravados a fazer várias repetições de diferentes atividades diárias, cujo objetivo é promover a reabilitação motora. Os vídeos foram adquiridos usando uma câmera ZED da StereoLabs. Posteriormente recorreu-se ao MediaPipe[†], um modelo baseado em redes neuronais, para a deteção e extração de um conjunto de marcadores a cada 2 segundos de vídeo. Estes marcadores, como exemplificados na Figura 2, correspondem a posições 2D $(u_{j,t}^{(i)}, v_{j,t}^{(i)})$ de várias partes do corpo humano j e podem ser ligados por arestas, permitindo a formação de um esqueleto representativo da pose do paciente no dado instante t.

Formalmente, o conjunto de dados a ser usado consiste em sequências de poses de 18 pacientes. Cada paciente executa o mesmo número de vezes os seguintes exercícios: i)E_1, escovar o cabelo; ii) E_2 lavar os dentes; iii) E_3 lavar a cara; iv) E_4 calçar as meias; e v) E_5 levantar os joelhos. Cada um dos exercícios é feito várias vezes por todos os pacientes. Adicionalmente, os exercícios E_1 e E_2 são repetidos o mesmo número de vezes para cada lado do corpo. Cada sequência $i \in \{0, .., 1259\}$ contém um número variável de poses. Cada pose adquirida no instante t é representada por um conjunto de $j \in \{0, ..., 32\}$ marcadores, cada um deles definido por uma posição $(u_{j,t}^{(i)}, v_{j,t}^{(i)})$. Ou seja, dada uma sequência i, cada pose num instante t corresponde ao seguinte vetor: $[u_{0,t}^{(i)}, v_{0,t}^{(i)}, u_{1,t}^{(i)}, v_{1,t}^{(i)}, ..., u_{32,t}^{(i)}, v_{32,t}^{(i)}]$.

Para além das sequências e da classe associada ao tipo de exercício, são também providenciadas no ficheiro de dados as seguintes informações: i) o ID do paciente; e ii) se na sequência ocorre compensação ao nível dos ombros, tronco e/ou cabeça. Os movimentos compensatórios são indesejados e levam a variações dentro do movimento esperado. Isto causa variabilidade dentro das várias repetições do mesmo paciente, assim como entre pacientes.

[†]https://github.com/google-ai-edge/mediapipe

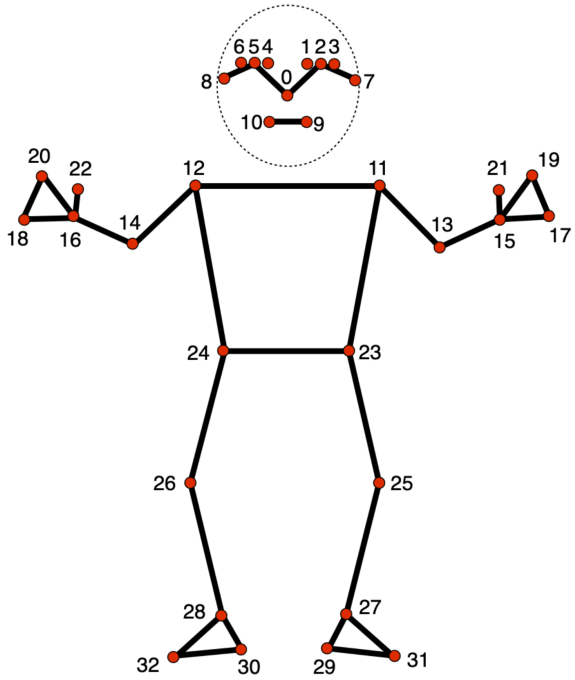
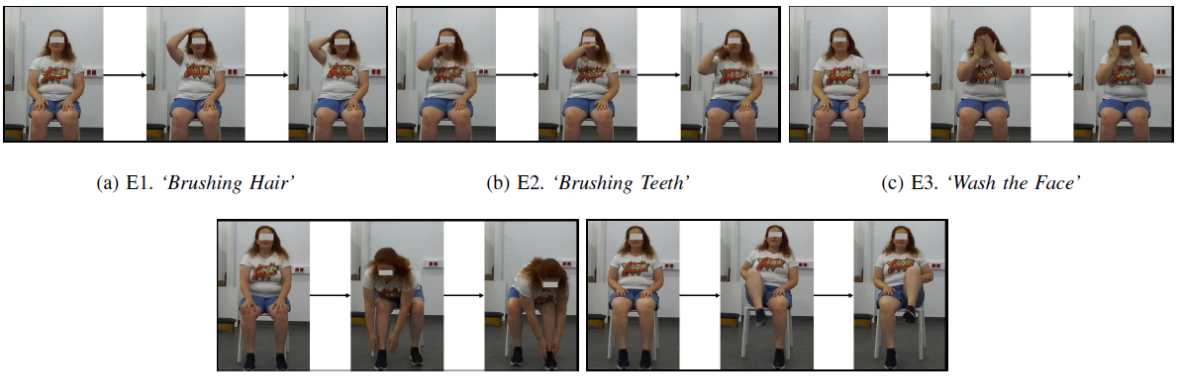


Figura 1: Esqueleto obtido com o MediaPipe Skeleton Keypoints. Cada número corresponde a um marcador e estes seguem a mesma ordem que no projeto.



(a) E1. *'Brushing Hair'* (b) E2. *'Brushing Teeth'* (c) E3. *'Wash the Face'*

(d) E4. *'Put on Socks'* (e) E5. *'Hip Flexion'*

Figura 2: Exemplo dos vários exercícios realizados.

Os dados encontram-se guardados no formato pickle (.pkl), podendo ser integrados no código através da biblioteca *pandas*, que criará uma matriz estruturada de dados. A navegação da matriz/tabela de dados gerada é em tudo semelhante à que foi adoptada na 1ª entrega do laboratório (Aprendizagem não Supervisionada).

O ficheiro `funcoes_aux_proj_apr_supervisionada.py` contém várias funções auxiliares a serem usadas na resolução do mini-projecto.

Enunciado

Grupo: **Turno:**

Nome: **IST ID:**

Nome: **IST ID:**

Parte I - Regressão Linear

Esta secção tem como objetivo explorar a estimação dos coeficientes de uma regressão linear, com aplicação num modelo prático auto-regressivo da forma:

$$y(t) = y(t-1)^T \beta + \beta_0, \tag{1}$$

onde $\beta \in \mathbb{R}^d$ é o vector de coeficientes da regressão e β_0 é o intercept.

1. **Implementação:** Nesta secção iremos implementar um estimador para os parâmetros β e β_0 do modelo auto-regressivo usando o método dos mínimos quadrados. **O código desta questão deverá ser guardado num ficheiro Python chamado** `reg_linear.py`.

   R1.a) Escreva uma função em Python que dada uma matriz de dados X e o vector the saídas correspondentes Y estima os coeficientes de uma regressão linear. A função deve implementar a estimação de $\hat{\beta}$ e $\hat{\beta}_0$ recorrendo às equações normais. **Não use a função de estimação disponível no scikit-learn!**

   R1.b) Escreva código que selecione todas as sequências do E_5 do paciente com o ID 4. Depois separe as sequências, guardando as últimas 20% para teste. **Não altere a ordem das sequências face à dada originalmente.**

   R1.c) Organize todas as poses das sequências de treino numa matriz X e as saídas Y, de forma a estimar os coeficientes do modelo auto-regressivo.
   Nota: Pretende-se que o modelo use apenas a pose anterior para prever a seguinte, pelo que X_{treino} e Y_{treino} devem ser construídos desta forma. Adicionalmente, deve acrescentar uma coluna de 1s a X para contemplar o β_0.

   R1.d) Repita a alínea anterior, construindo as matrizes X_{teste} e Y_{teste}

   R1.e) Escreva código para calcular a soma dos erros quadráticos (SSE) e o R^2 dados um $\hat{Y}$ estimado e os valores verdadeiros.
   Nota: Tenha em conta que a estimativa será uma pose e não um escalar, pelo que $\hat{y}^{(i)} \in \mathbb{R}^{66}$

2. **Estimação dos Coeficientes e Resultados:**

   R2.a) Use a função desenvolvida para estimar os coeficientes $\hat{\beta}$ e $\hat{\beta}_0$. Reporte as dimensões dos mesmos e comente.

   R2.b) Aplique o modelo estimado aos dados de treino. Reporte o SSE e o R^2 globais.

R2.c) Repita a questão anterior, mas agora usando as sequências de teste. Comente as diferenças. Em que *keypoints* é que o SSE é maior?

R2.d) Utilize a função fornecida `visualize_sequences` para comparar visualmente as sequências verdadeiras com as previstas. O que observa?

Parte II - Árvores de Decisão

Nesta secção vamos estudar o algoritmo das árvores de decisão, num problema de classificação multi-classe. Todas as sequências de um paciente serão utilizadas para treinar e avaliar árvores com diferentes níveis de profundidade. Iremos também experimentar o processo de extração de *features*/características de forma a representar as sequências.

3. **Implementação:** Nesta secção iremos implementar uma árvore de decisão, utilizando a estrutura de classes do Python[‡]. A árvore deverá ser implementada no ficheiro `arvore_decisao.py`, que já inclui algum código de apoio para a criação da classe "Node"(nó).

   R3.a) Complete o ficheiro `arvore_decisao.py` de forma a implementar um programa que treine e faça previsões com uma árvore de decisão. A árvore será construída tendo em conta

[‡] https://docs.python.org/3/tutorial/classes.html#

os seguintes critérios:

- As *features* a usar são numéricas. Isto significa que cada nó fará o *split* baseado na aplicação de um limiar (*threshold*) a uma *feature*.
- O critério da informação mútua será usado para definir o *split* nos nós. Os splits só devem ser feitos enquanto a informação mútua for positiva.
- A classificação numa folha é feita por votação. Em caso de empate, deve ser escolhida a classe correspondente ao maior valor.
- Se várias *features* tiverem o mesmo valor de informação mútua, escolher aquela que aparece primeiro nas colunas.
- Não deixar crescer a árvore para além do valor de `profundidade_maxima` definido pelo utilizador.
- Para o cálculo da informação mútua, será útil definir uma função que calcula a entropia.
- Uma árvore deve ser vista como um conjunto de nós. Por isso, será útil definir uma função para treinar um único nó e depois chamá-la recursivamente para fazer crescer a árvore.

R3.b) Escreva código para retirar todas as sequências do **paciente 4** e separe-as em conjuntos de treino e teste, usando as 20% últimas sequências como teste.
Nota: Repare que terá de converter os nomes dos exercícios (E1-E5) em classes. Poderá substituir os nomes por números de 0 a 4. Esta conversão pode ser utilizada nas partes seguintes do projecto.

R3.c) Escreva uma função que aplique a função `extrair_atributos` às várias sequências de treino e teste, de forma a criar as matrizes X_{train} e X_{test}.

R3.d) Investigue na biblioteca do `scikit-learn` e identifique as funções que permitem calcular: i) a matriz de confusão e ii) a exatidão balanceada (*balanced accuracy*). Escreva uma função que calcule estas métricas dado um vector de classes previstas e outro com as classes verdadeiras. **Nota:** Esta função será útil nas secções seguintes também.

4. **Treino da Árvore e Resultados**

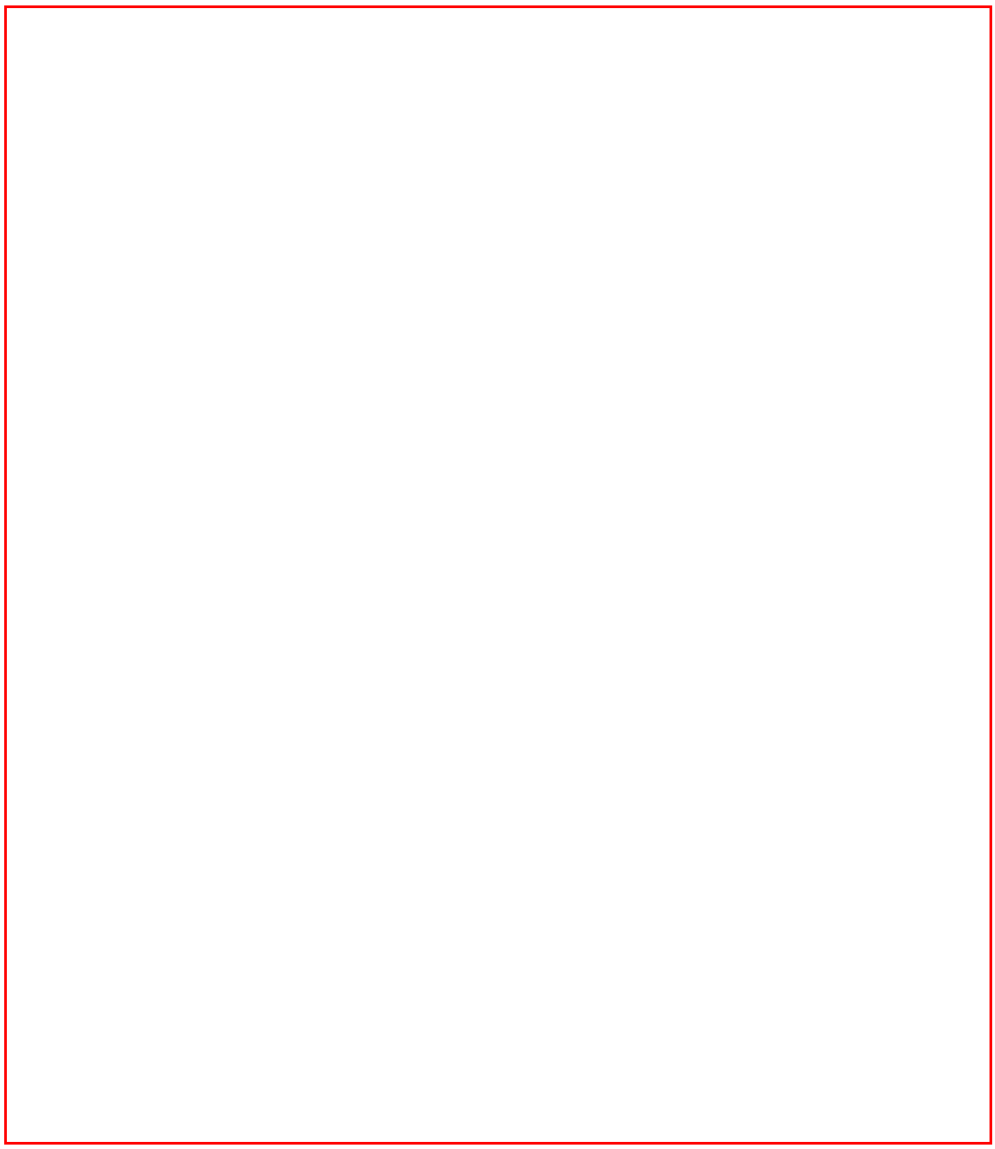
R4.a) Utilize a função implementada para treinar uma árvore de decisão. Faça variar a profundidade máxima entre 1 e 5. Para cada um destes valores, reporte a exatidão balanceada (*balanced accuracy*) para os conjuntos de treino e teste.



R4.b) Calcule as matrizes de confusão para os conjuntos de treino e teste e guarde as mesmas em figuras. Comente as mesmas e os resultados obtidos na alínea anterior.



R4.c) Utilize a função `imprimir_arvore` para visualizar a árvore obtida com profundidade 4. Cole o resultado abaixo e comente a árvore obtida.



Parte III - Naive Bayes

Nesta secção vamos estudar o classificador Naive Bayes em dois cenários distintos: i) usando as *features* calculadas anteriormente; e ii) recorrendo a uma abordagem na qual representamos as sequências como um *Bag-of-Poses*. Nesta última abordagem, iremos usar o algoritmo *k-means* para obter um dicionário de poses com o qual iremos representar cada sequência como um histograma. Em detalhe, cada pose será sumarizada num vector de dimensão K que contém a frequência de cada uma das poses na sequência.

A abordagem descrita acima é inspirada na abordagem *Bag-of-Words* que se tornou popular antes dos modelos recentes de visão e linguagem para representar dados de imagem e texto, pela sua simplicidade e interpretabilidade.

5. **Features manuais**

R5.a) Mantendo os conjuntos de treino e teste usados na secção anterior, use a função `GaussianNB` do `scikit-learn` para treinar um classificador Naive Bayes. Calcule e reporte a *balanced*

accuracy para os conjuntos de treino e teste, assim como a matriz de confusão. Comente e compare com os resultados obtidos com a melhor árvore de decisão.



6. **Bag-of-Poses:** Escreva código que lhe permita executar os seguintes passos:

- Aplicar o algoritmo *k-means* com um número de centróides definido pelo utilizador a todas as poses das sequências de treino. **Nota:** Poderá reutilizar o código de *k-means* feito para a primeira entrega de laboratório.
- Para cada sequência associar poses aos centróides e construir um histograma da frequência dos centróides. **Nota:** Ver a função `histogram` da biblioteca `numpy`.
- Treinar e avaliar um classificador de Naive Bayes usando a representação de *Bag-of-Poses*.

R6.a) Treine um classificador de Naive Bayes para classificar os exercícios, fazendo variar o número de centróides $K \in \{2, 5, 10, 15\}$. Reporte os resultados da *balanced accuracy* para os conjuntos de treino e teste e comente o que observa.



R6.b) Obtenha a matriz de confusão para um valor de K à sua escolha, justificando a mesma. Compare a matriz obtida com aquela que se obteve com as *features manuais* e com a melhor árvore de decisão da secção anterior.



Parte IV - Regressão Logística

Nesta secção vamos estudar um classificador linear multiclasse baseado na regressão logística. Será pedido que implemente o treino de uma regressão logística recorrendo ao algoritmo do gradiente, nomeadamente o SGD. Como no caso da árvore de decisão, iremos usar a estrutura de classes do Python.

7. **Implementação:** Modifique o ficheiro `rl.py` de forma a completar as funções em falta, possibilitando o treino de uma regressão logística e a aplicação da mesma a um conjunto de dados para fazer previsões. A implementação deve estar de acordo com as seguintes instruções:

- A regressão logística será aplicada num problema multiclasse; por esse motivo, a não linearidade a aplicar à saída será a *softmax* - este código já é dado.
- Deve ser possível modificar o *learning rate* e o número de épocas de treino.
- Inclua um *intercept/offset* que deve ser estimado conjuntamente com os restantes parâmetros.
- Todos os parâmetros devem ser inicializados em 0.
- O treino deve ser feito recorrendo ao **SGD**. **Não baralhe os exemplos de treino entre épocas.** Estes devem ser passados sempre pela mesma ordem.
- Em cada época, guarde o valor da função de custo, de forma a que seja possível posteriormente visualizar a evolução da mesma num gráfico.
- O modelo deve devolver o exercício mais provável para cada sequência. Durante o treino será necessário converter o vector de classes Y para um *one-hot-encoding*, de forma a ter as mesmas dimensões que a saída da regressão.
- Considere a hipótese de normalizar os dados de treino e teste usando o `StandardScaler` do `scikit-learn` ou outro método.
- **Não deve usar a função da regressão logística já implementada no scikit-learn!**

R7.a) Treine dois modelos de regressão logística, um usando as *features manuais* e outro usando as features obtidas pelo *Bag-of-Poses*. Para ambos os casos, use um *learning rate* de 0.01 e 50 épocas. Reporte a *balanced accuracy* obtida nos conjuntos de treino e teste para ambos os casos, e obtenha as matrizes de confusão.



R7.b) Repita a alínea anterior, mas agora fazendo variar o *learning rate* no seguinte intervalo $\{0.01, 0.001, 0.0001, 0.00001\}$. Faça um gráfico para cada um dos métodos de extração de *features* em que no eixo das abcissas tem o número de épocas e no das ordenadas o valor da função de custo (entropia cruzada). Obtenha também as matrizes de confusão para cada par de extractor de *features*/*learning rate*. Comente o que observa.



Parte V - Redes Neuronais e Generalização de Modelos

Nesta secção iremos trabalhar com todos os pacientes, desenvolvendo modelos baseados em redes neuronais, uma vez que estes tendem a requerer um maior volume de dados. Será também importante implementar nesta secção abordagens que permitam a seleção de modelos, hiperparâmetros, de forma a garantir a generalização dos métodos.

A implementação das redes neuronais deverá ser feita recorrendo ao PyTorch[§], pelo que se recomenda a leitura do tutorial, de forma a garantir que o desenho, treino e avaliação dos modelos estão corretos.

8. **Implementação de um MLP:** Nesta parte iremos implementar um perceptrão multicamada para classificar os vários exercícios dadas as *features* manuais utilizadas anteriormente. Deve criar código que verifique os seguintes critérios:

- Selecionar todas as sequências dos pacientes 1 a 13 para treino e 14 a 18 para teste. De seguida, deve ser possível usar a função `extrair_atributos` para converter as sequências em vectores de features, assim como a abordagem do *Bag-of-Poses* implementada para o classificador Naive Bayes.
- Comparar diferentes arquitecturas para o MLP, nomeadamente número de camadas e de unidades/neurónios e selecionar a melhor para aplicar ao conjunto de teste. Também pode ser boa política comparar diferentes valores de *learning rate*. **Nota:** O modelo irá ser aplicado a um conjunto diferente de pacientes dos que foram usados para treino, por isso será necessário seguir um protocolo de seleção de modelos apropriado.
- Usar a entropia cruzada como função de custo.
- Pode testar múltiplas variantes do algoritmo do gradiente, nomeadamente aquelas que usem termos de momentum e/ou *learning rates* adaptativos. **Todavia, deve compreender os algoritmos que está a usar.**
- Gravar o melhor modelo após o treino, de forma a poder reutilizar o mesmo.
- Para evitar o *overfitting*, considerar o uso do critério de *Early Stopping*.[¶]

R8.a) Defina uma arquitetura de MLP para prever o tipo de exercício usando os dois métodos de extração de *features*. Indique a melhor configuração de treino para cada caso. Descreva também a gama de hiperparâmetros que usou e como selecionou as melhores arquitecturas e configurações de treino.

[§] https://docs.pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html
[¶] https://docs.pytorch.org/ignite/v0.5.4/generated/ignite.handlers.early_stopping.EarlyStopping.html

R8.b) Aplique ambas as arquitectura aos dados de teste. Calcule a *balanced accuracy* e as matrizes de confusão. Discuta se há um desvio face ao esperado com os dados de treino e tente explicar porquê.

9. **Implementação de uma CNN:** Vamos agora trabalhar com as sequências no seu formato original, preservando os keypoints e a organização temporal das poses. Desta forma, será necessário escrever código que:

- Converta as sequências de poses em "imagens". Para cada sequência de treino e teste, deverá concatenar as mesmas ao longo das colunas, de forma a obter uma matriz de dimensões $66 \times T$, onde T é o comprimento da sequência.

Nota: Uma vez que as sequências terão comprimentos diferentes, deve ser feito padding com zeros nas sequências de comprimento mais curto para que todas as matrizes tenham um número de colunas igual ao da sequência de maior dimensão.

- À semelhança do MLP, deve ser possível comparar os hiperparâmetros da CNN - número de camadas, número e tamanho dos filtros, learning rate, etc.
- Neste caso poderá ser relevant usar um treino em mini-batch em vez de passar os dados todos ou usar o SGD, uma vez que demorará mais tempo a correr.
- Usar a entropia cruzada como função de custo.
- Gravar o melhor modelo após o treino, de forma a poder reutilizar o mesmo.
- Para evitar o *overfitting*, considerar o uso do critério de *Early Stopping*.
- Recomenda-se o uso do Google Colab para o treino das CNNs.

R9.a) Defina uma configuração de CNN para prever o tipo de exercício. Indique que hiperparâmetros testou, assim como o protocolo de treino e diga qual a configuração escolhida.



R9.b) Indique qual a *balanced accuracy* obtida nos pacientes de teste e guarde uma imagem com a matriz de confusão. Comente, comparando com os resultados do MLP. O que consegue concluir sobre os dois modelos?



R9.c) Os algoritmos de redes neuronais são muitas vezes considerados caixas negras, ao contrário de modelos como as árvores de decisão, uma vez que não são transparentes nem interpretáveis. Neste sentido, têm vindo a ser desenvolvidos mecanismos que pretendem corrigir esta limitação. Uma linha de métodos foca-se em providenciar explicações *post-hoc*, i.e., após o modelo estar treinado. Um destes métodos chama-se Grad-CAM,[‖] que visa identificar regiões na imagem que apresentam maior peso/importância para a previsão. Estas regiões são identificadas através de mapas de saliência, que podem ser visualizados. Nesta alínea iremos aplicar o GRAD-CAM a algumas sequências corretamente e incorretamente classificadas, tentando interpretar as decisões do modelo. Desta forma, deve selecionar pelo menos 5 casos bem classificados e 5 casos mal classificados do conjunto de teste, que sejam representativos dos vários exercícios e aplicar a função `grad_cam` aos mesmos. Depois use a função `plot_gradcam` para visualizar os resultados. Guarde as figuras obtidas e discuta o que observa.
Nota: O GRAD-CAM permite ver as activações em várias camadas da rede. Sugerimos que inspecione, pelo menos, as activações antes da operação de *pooling* em que se perde a componente espacial. Poderá também inspecionar mais de 5 exemplos para cada cenário.

[‖] Grad-CAM: Visual Explanations From Deep Networks via Gradient-Based Localization Ramprasaath R. Selvaraju, et al. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618-626



Parte VI - Pergunta Aberta

Um dos maiores desafios da aprendizagem supervisionada é a necessidade de anotar os dados de treino, de forma a gerar pares $(x^{(i)}, y^{(i)})$ que possam suportar a aprendizagem dos modelos. Muitas vezes a aquisição destas anotações representa um grande esforço para os anotadores, pelo que se opta pela produção daquilo que chamamos anotações "fracas". Estas anotações não nos dão um mapeamento detalhado da relação entre as características $x^{(i)}$ e a tarefa que queremos resolver, mas correspondem a um passo intermédio. Um exemplo seria querermos detectar/localizar gatos em imagens, mas apenas termos acesso a uma anotação global a indicar que imagens contêm gatos e quais não contêm. Daqui, um modelo é treinado para inferir a localização dos gatos a partir da anotação global. Este tipo de aprendizagem chama-se **Aprendizagem Fracamente Supervisionada** e é aquilo que se pretende que os alunos estudem nesta questão.

Um dos métodos mais populares para abordar o problema da aprendizagem fracamente supervisionada é a formulação de **Multiple Instance Learning**.[**]Tendo por inspiração o artigo referido e o trabalho desenvolvido nas restantes perguntas do laboratório, pretende-se que seja implementada uma abordagem que permita anotar individualmente as poses das várias sequências quanto:

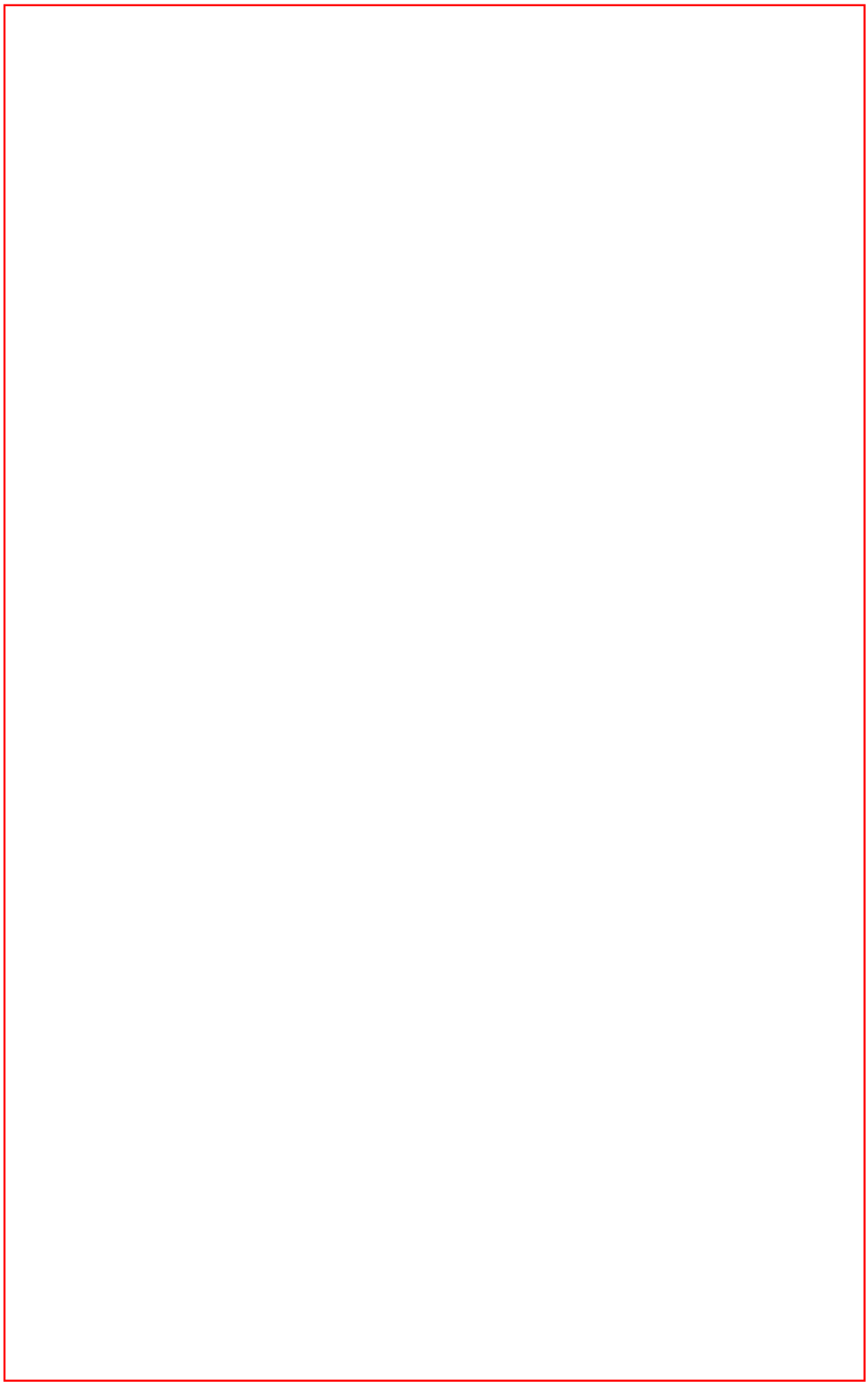
1. A se existe ou não compensação nos ombros, tronco e cabeça naquele instante/pose;
2. Quais os *keypoints*/marcadores em que a compensação se observa.

As seguintes regras devem ser adoptadas:

- Deve ser utilizada a mesma partição de pacientes para treino e teste que na Parte V do enunciado para o treino e avaliação final do modelo.
- Deve ser reportada a *balanced accuracy* e a matriz de confusão obtida para o conjunto de teste, obtidas ao nível da sequência, para a abordagem desenvolvida. **A tarefa de classificação é a previsão dos vários tipos de compensação ao nível da sequência, sendo as anotações ao nível da pose e marcadores obtidas como *output* adicional.**
- Poderão usar quaisquer algoritmos leccionados na cadeira, para além daqueles abordados no enunciado. **Não é possível usar algoritmos de aprendizagem supervisionada que não tenham sido leccionados.** A abordagem pode conter uma combinação de algoritmos.

10. Descreva sucintamente a abordagem desenvolvida, reporte as métricas de avaliação pedidas acima e indique qual o número de poses que foi anotada com cada um dos tipos de compensação. Discuta se o número de poses classificada como tendo cada tipo de compensação é razoável. Inspecione também os *keypoints* que foram mais comunmente associados a cada tipo de compensação e discuta se obteve o resultado esperado ou não.

[**]Waqas, Muhammad, Syed Umaid Ahmed, Muhammad Atif Tahir, Jia Wu, and Rizwan Qureshi. "Exploring multiple instance learning (MIL): A brief survey."Expert Systems with Applications 250 (2024): 123893.



A Declaração de Honra

Cada grupo deve preencher e assinar a seguinte declaração.

Declaração de Honra

Nós, membros do grupo [número do grupo], identificados abaixo

[Aluno 1], No. [xxxxxx]

[Aluno 2], No. [xxxxxx]

Declaramos por nossa honra que este trabalho, realizado ao abrigo da cadeira de Introdução à Aprendizagem Automática, é resultado do trabalho dos seguintes membros do grupo: [membros].

As contribuições individuais são:

[Aluno 1] – [breve descrição da contribuição]

[Aluno 2] – [breve descrição da contribuição]

Declaramos que este trabalho é original e que foi feito segundo os regulamentos académicos.

Nome, Número e	Assinatura
_____	_____
_____	_____
_____	_____

Data: _____

B Declaração do Uso de Inteligência Artificial

Cada aluno deve preencher e assinar uma cópia individual desta declaração.

Declaração do Uso de Inteligência Artificial

Nome do Aluno: _____

Número do Aluno: _____

Declaro que todas as decisões e conteúdo técnico deste laboratório são da minha autoria.

- [listar ferramenta] para [finalidade]

Declaro que compreendo completamente o funcionamento de todas as componentes do projeto, mesmo as que foram desenvolvidas com o apoio das ferramentas de IA listadas acima."

Data: _____ Assinatura: _____